

第1回：SQL基礎（SELECT/INSERT）～クエリで「データの旅」をつかむ～

1. 本日の内容の概要（最初の確認）

今日は、**SQLでDBにお願いする手順（型）**を覚える回です。

最初は「できるだけ短い手順」を**写経→実行→確認**で回します。

2. 学習目標（手順を覚える）

この回では、SQLを使って「見る／足す／変える／消す」の流れを一通りたどれることを目指します。

▼ SELECT・INSERT・DELETE・UPDATEについてそれぞれを説明できる

それぞれのSQLが「どんなときに、どんな働きをするか」を日本語で説明できる。

- **SELECT**: 必要な情報を抽出して任意のカラムを表示させる。
- **INSERT**: 任意の情報を登録できる。
- **DELETE**: 任意の情報を削除できる。
- **UPDATE**: 任意の情報を更新できる。

▼ 手順の型

次の順番で一通りたどれること。

INSERT ▶ SELECT ▶ UPDATE ▶ SELECT ▶ DELETE ▶ SELECT

3. 写経パート（手続きとして覚える）

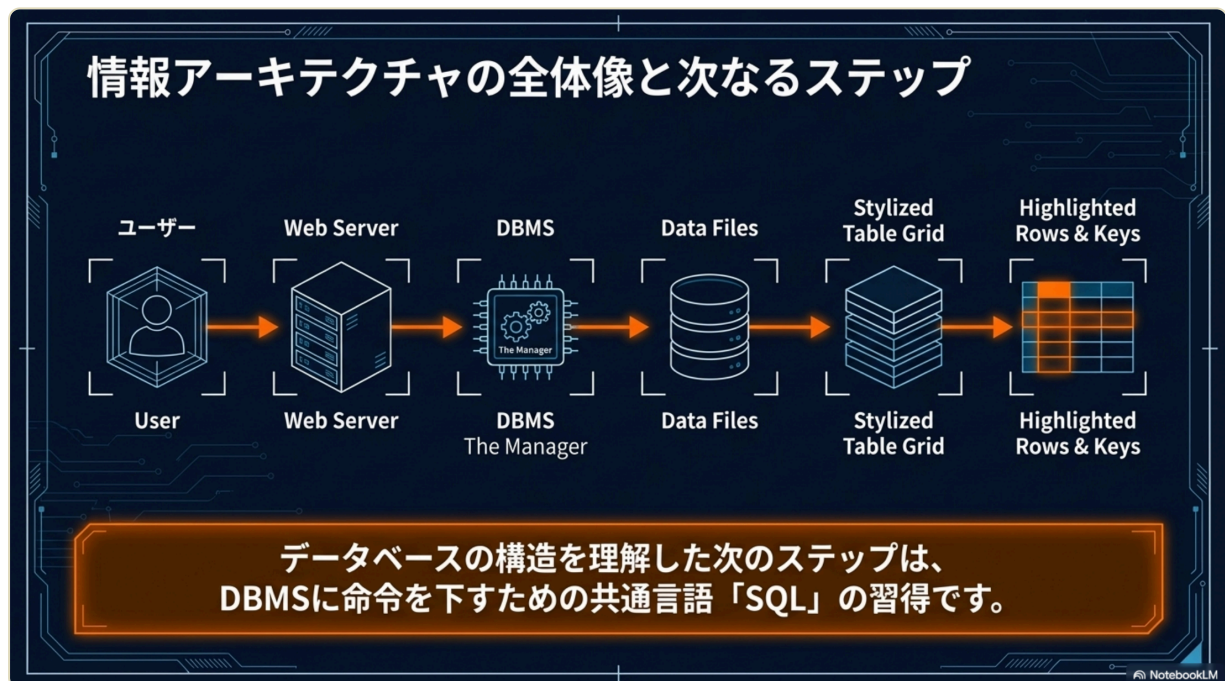
3-1. 用語の確認（最小）

- **カラム（列）**：項目。例） `item_name`、 `unit_price`
- **ロウ（行）**：1件分のデータ（レコード）。
- **主キー（Primary Key）**：行を一意に特定するための値。例） `item_id`
- **引用（クオート）**：文字列は `'水'` のように `'（シングルクォート）` で囲む。数値は通常囲まない。

SQL

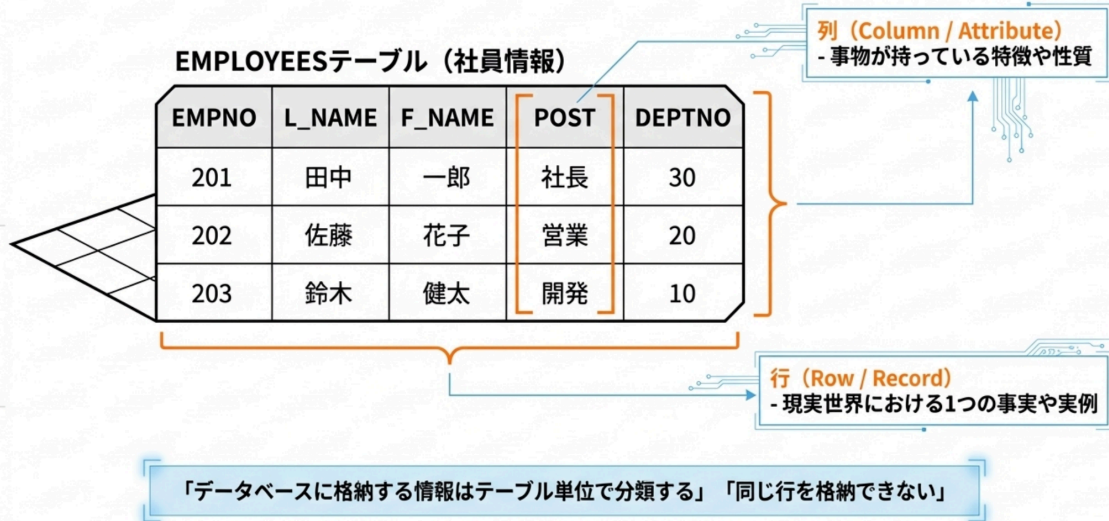
この回で扱う **SQLの役割と基本の書き方** を、図とあわせて整理しておきます。iPadで表示して、上から書き込みながら説明できるようにまとめています。

SQLの位置づけ：アプリやツールの代わりに、DBMSに命令を出すための共通言語がSQLです。



テーブル（列と行）：列＝項目、行＝1件分のデータです。

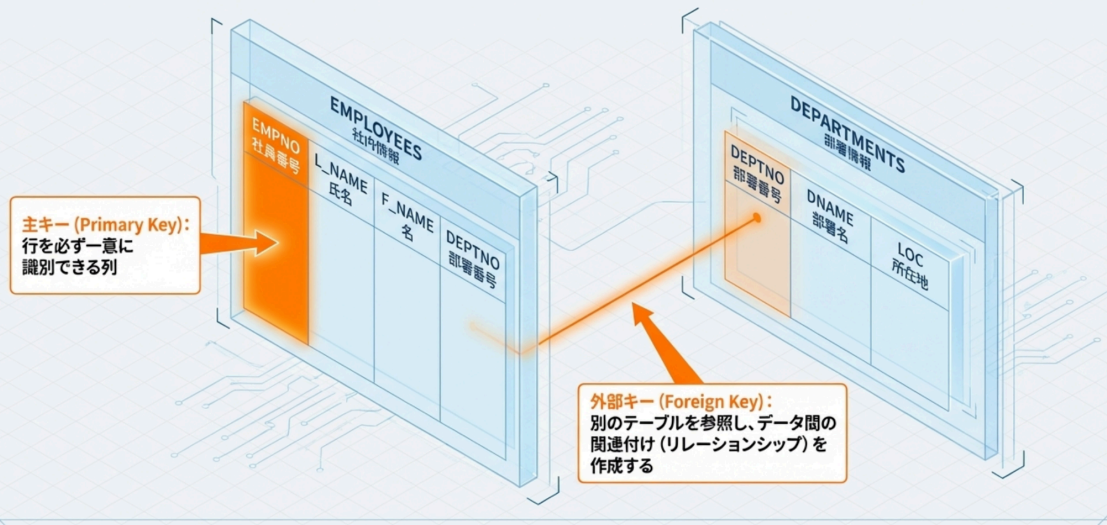
リレーショナルデータベースの基本単位「テーブル」の解剖学



© NotebookLM

主キーと外部キー (リレーション) : テーブル同士のつながりを作るためのキーです。

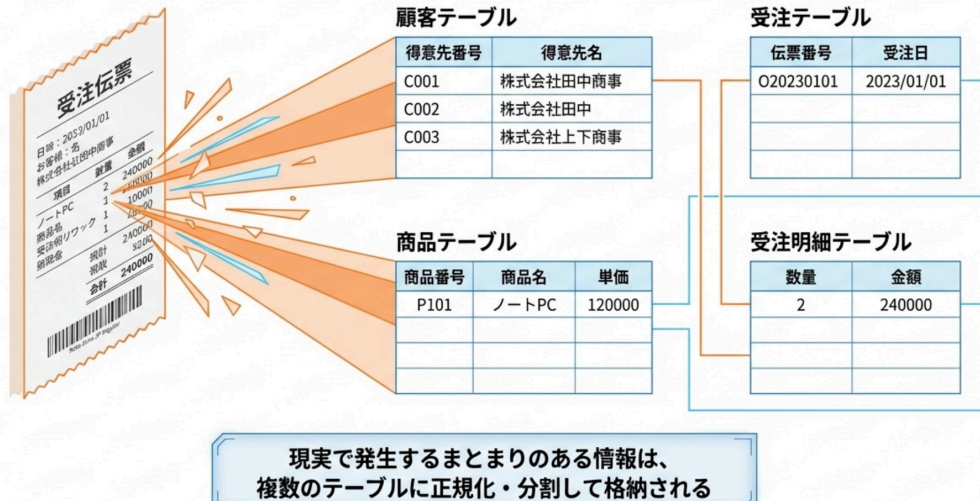
データを一意に特定し、テーブル同士を繋ぐ「キー」の役割



© NotebookLM

正規化 (概要) : 現実のまとまりある情報を、意味ごとに分割してテーブルに整理します。

現実世界の複雑な情報は、意味ごとに分割して記録される



SQLの基本文 (SELECT / WHERE / ORDER BY)

SELECT : どのカラムを「見る」か決める

SELECT は「どのカラムを、どのテーブルから取り出して表示するか」を決める文です。SELECT * は「全部のカラム」、カンマ区切りで列名を書くと「必要なカラムだけ」取り出せます。

-- すべてのカラムを表示

```
SELECT * FROM items;
```

-- 一部のカラムだけ表示

```
SELECT item_id, item_name  
FROM items;
```

- * は最初の練習では便利だが、本番では**必要なカラムだけ**列挙するのが基本。
- カラム名の順番は、画面に出てくる列の順番にもなる。

WHERE : 条件でしぼり込む

WHERE は「どの行を対象にするか」を決める条件の文です。WHERE 列 演算子 値 の形で書きます。

```
SELECT *  
FROM items  
WHERE category = 'drink';
```

主な演算子 (数値・文字に共通)

- = : 等しい (例: unit_price = 100)

- `<` / `<=` : より小さい / 以下
- `>` / `>=` : より大きい / 以上
- `<>` : 等しくない (`!=` を使うDBもある)

BETWEEN : 範囲でしぼり込む

```
-- 100円以上 150円以下
SELECT *
FROM items
WHERE unit_price BETWEEN 100 AND 150;
```

- BETWEEN は「両端を含む」範囲（以上かつ以下）になる。
- 日付でも同じ書き方ができる（例： `WHERE order_date BETWEEN '2025-04-01' AND '2025-04-30'` ）。

LIKE とあいまい検索

```
-- 「茶」で終わる飲み物
SELECT *
FROM items
WHERE item_name LIKE '%茶';
```

- `%` : 0文字以上の「なんでも」（前方一致・後方一致・部分一致に使う）。
- `_` : 1文字だけ「なんでも」。

IS NULL : NULL（まだ値が入っていない）の判定

```
SELECT *
FROM items
WHERE some_column IS NULL;

SELECT *
FROM items
WHERE some_column IS NOT NULL;
```

- NULL は「0」でも「空文字」でもなく、「まだ値が入っていない・わからない」という特別な値。
- NULL との比較は `=` ではなく `IS` を使う（`= NULL` と書くとヒットしない）。

AND / OR : 条件を組み合わせる

```
SELECT *
FROM items
WHERE category = 'drink'
AND unit_price > 100;
```

- **AND** : 両方の条件を満たす行だけ残す。

- **OR** : どちらか一方でも満たす行を残す（条件がゆるくなりやすいので注意）。

ORDER BY : 並び順を決める

ORDER BY は「結果をどの順番で並べるか」を決める文です。書く位置は **WHERE のあと** になります。

```
-- 単価の高い順（降順）
SELECT *
FROM items
ORDER BY unit_price DESC;

-- スナックだけを、安い順（昇順）に並べる
SELECT *
FROM items
WHERE category = 'snack'
ORDER BY unit_price ASC;
```

- **ASC** : 昇順（小さい→大きい）。省略するとASCになる。
- **DESC** : 降順（大きい→小さい）。
- 複数カラムで並べたいときは、カンマでつなぐ（例 : `ORDER BY category ASC, unit_price DESC`）。

次は、データを変える3文

- **INSERT** : 行を追加。例 : `INSERT INTO items ( ... ) VALUES ( ... );` (`item_id` が重複すると失敗しやすい)
- **UPDATE** : 行を更新。基本は `UPDATE items SET ... WHERE item_id = ...;` (**WHEREなし**は全行更新になりやすい)
- **DELETE** : 行を削除。基本は `DELETE FROM items WHERE item_id = ...;` (**WHEREなし**は全削除になりやすい)

```
-- INSERT (行を追加)
INSERT INTO items (item_id, category, item_name, unit_price)
VALUES (305, 'drink', 'サンプル2', 140);

-- UPDATE (1行だけ変更 : WHEREで狙う)
UPDATE items
SET unit_price = 110
WHERE item_id = 101;

-- DELETE (1行だけ削除 : WHEREで狙う)
DELETE FROM items
WHERE item_id = 202;
```

3-2. MySQL への接続と初期データ投入

MySQL の接続手順・DB選択コマンド・テーブルへのSQL流し込みは、「**環境構築：MySQL のセットアップとDB接続手順**」（Html-env-01）を参照してください。

接続コマンド（`mysql -u root -p`）、SHOW DATABASES / USE / SHOW TABLES の使い方、ハンズオンSQL（items・depts・emps の3テーブル）はそちらに掲載しています。

3-3. INSERT / SELECT / UPDATE / DELETE の型（最小）

この回は、次の並びを「同じ順番で」たどれることがゴールです。

INSERT ▶ SELECT ▶ UPDATE ▶ SELECT ▶ DELETE ▶ SELECT

```
-- INSERT（行を追加）
INSERT INTO items (item_id, category, item_name, unit_price)
VALUES (999, 'drink', 'サンプル', 100);

-- SELECT（見る：条件/並び/行数）
SELECT item_id, category, item_name, unit_price
FROM items
WHERE category = 'drink'
ORDER BY item_id
LIMIT 10;

-- UPDATE（1行だけ変更：WHEREで狙う）
UPDATE items
SET unit_price = 110
WHERE item_id = 101;

-- DELETE（1行だけ削除：WHEREで狙う）
DELETE FROM items
WHERE item_id = 202;
```

SQLの書き方には、流れとしての「順番の型」があります。下のテンプレートは、その流れを整理したものです。

【手順テンプレート】

- 1) 初期データを入れる (INSERT)
- 2) 一覧を確認する (SELECT)
- 3) 1行だけ変更する (UPDATE)
- 4) もう一度確認する (SELECT)
- 5) 1行だけ消す (DELETE)
- 6) 最後に確認する (SELECT)

【メモテンプレート (結果を書き写す)】

SELECT結果：行数 = ____ 行

変更した内容： unit_price を ____ → ____ にした

削除した行： item_id = ____

最終の行数： ____ 行 (増えた？減った？)

3-4. 写経の練習 (問題 + 解答)

次の日本語をもとに、SQLを書いてみましょう。まず自分で考えてから、ヒントや解答を確認してください。

最初に流し込むSQL (テーブル作成 + 初期データ)

USE lesson; のあと、環境構築ページと同じ **items 定義 (10件)** を使います。復習ページでも同じSQLで初期化できます。

この枠のSQLをコピーして、手元のツールに貼り付けて実行してください：

```
DROP TABLE IF EXISTS items;

CREATE TABLE items (
    item_id    INT          NOT NULL,
    category   VARCHAR(20)  NOT NULL,
    item_name  VARCHAR(50)  NOT NULL,
    unit_price INT          NOT NULL,
    PRIMARY KEY (item_id)
```

問題1： itemsテーブルの全データをすべてのカラムで表示するSQLを書いてください。

▼ ヒント

SELECT * はすべてのカラムを取り出す書き方です。FROM のあとにテーブル名を書きます。

▼ 解答例

```
SELECT * FROM items;
```

マウスで全選択して貼り付け欄へドラッグしてください：

```
SELECT * FROM items;
```

問題2：itemsテーブルから `item_id` と `item_name` の2カラムだけを表示するSQLを書いてください。

▼ ヒント

SELECT のあとに取り出したいカラム名をカンマ区切りで書きます。`*` を使わない場合、必要な列だけを指定できます。

▼ 解答例

```
SELECT item_id, item_name  
FROM items;
```

マウスで全選択して貼り付け欄へドラッグしてください：

```
SELECT item_id, item_name  
FROM items;
```

問題3：category が `'drink'` の商品をすべてのカラムで表示するSQLを書いてください。

▼ ヒント

WHERE 列名 = '値' の形で条件を書きます。文字列はシングルクォートで囲みます。

▼ 解答例

```
SELECT *  
FROM items  
WHERE category = 'drink';
```

マウスで全選択して貼り付け欄へドラッグしてください：

```
SELECT *  
FROM items  
WHERE category = 'drink';
```

問題4： item_id が 200 より小さい商品をすべてのカラムで表示するSQLを書いてください。

▼ ヒント

数値の比較には <（未満）や <=（以下）が使えます。数値はシングルクォート不要です。

▼ 解答例

```
SELECT *  
FROM items  
WHERE item_id < 200;
```

マウスで全選択して貼り付け欄へドラッグしてください：

```
SELECT *  
FROM items  
WHERE item_id < 200;
```

問題5： itemsテーブルの全商品を、unit_price の高い順に並べて表示するSQLを書いてください。

▼ ヒント

`ORDER BY 列名 DESC` で降順（大きい順）に並べられます。昇順（小さい順）は `ASC`（省略可）です。

▼ 解答例

```
SELECT *  
FROM items  
ORDER BY unit_price DESC;
```

マウスで全選択して貼り付け欄へドラッグしてください：

```
SELECT *  
FROM items  
ORDER BY unit_price DESC;
```

問題6：category が 'snack' の商品を、unit_price の安い順に並べて表示するSQLを書いてください。

▼ ヒント

`WHERE` で絞り込んだあとに `ORDER BY` を書きます。書く順番は **WHERE** → **ORDER BY** です。

▼ 解答例

```
SELECT *  
FROM items  
WHERE category = 'snack'  
ORDER BY unit_price ASC;
```

マウスで全選択して貼り付け欄へドラッグしてください：

```
SELECT *  
FROM items  
WHERE category = 'snack'  
ORDER BY unit_price ASC;
```

問題7 : item_id が 101 の商品だけを表示するSQLを書いてください。

▼ ヒント

主キー (item_id) を指定すると、1件だけを確実に取り出せます。これが「1件特定」の基本パターンです。UPDATE・DELETE の前に必ず確認に使います。

▼ 解答例

```
SELECT *  
FROM items  
WHERE item_id = 101;
```

マウスで全選択して貼り付け欄へドラッグしてください：

```
SELECT *  
FROM items  
WHERE item_id = 101;
```

問題8 : unit_price が 120 以上の商品の item_name と unit_price だけを表示するSQLを書いてください。

▼ ヒント

「以上」は $\geq$ (以下は $\leq$) です。「より大きい」は $>$ 、「未満」は $<$ です。カラムは取り出したい列だけ指定します。

▼ 解答例

```
SELECT item_name, unit_price
FROM items
WHERE unit_price >= 120;
```

マウスで全選択して貼り付け欄へドラッグしてください：

```
SELECT item_name, unit_price
FROM items
WHERE unit_price >= 120;
```

問題9：category が 'drink' で、かつ unit_price が 100 より大きい商品を表示するSQLを書いてください。

▼ ヒント

複数の条件を組み合わせるには AND を使います。「どちらか一方」の場合は OR です。

▼ 解答例

```
SELECT *
FROM items
WHERE category = 'drink'
AND unit_price > 100;
```

マウスで全選択して貼り付け欄へドラッグしてください：

```
SELECT *
FROM items
WHERE category = 'drink'
AND unit_price > 100;
```

問題10：item_id が 200 以上の商品の item_name を、item_id の降順で表示するSQLを書いてください。

▼ ヒント

WHERE と ORDER BY を組み合わせるときは、WHERE を先に入ります。SELECT のあとには取り出したいカラムだけ指定します。

▼ 解答例

```
SELECT item_name
FROM items
WHERE item_id >= 200
ORDER BY item_id DESC;
```

マウスで全選択して貼り付け欄へドラッグしてください：

```
SELECT item_name
FROM items
WHERE item_id >= 200
ORDER BY item_id DESC;
```


4. 実戦演習（最小：10分）

最小タスク（10分で必ず終える）

1. 初期データ投入SQLを貼り付けて実行する（下の「SQL貼り付け欄」）。
2. `SELECT` で一覧を出す。
3. `UPDATE` を1回だけ実行して、単価を1つ変える。
4. もう一度 `SELECT` を実行して、変わったことを確認する。
5. `DELETE` を1回だけ実行して、1行消す。
6. 最後に `SELECT` を実行して、減ったことを確認する。

初期データ投入SQL（コピペ用）

この枠のSQLをコピーして、手元のツールに貼り付けて利用できます。

```
DROP TABLE IF EXISTS items;

CREATE TABLE items (
  item_id    INT          NOT NULL,
  category   VARCHAR(20)  NOT NULL,
  item_name  VARCHAR(50)  NOT NULL,
  unit_price INT          NOT NULL,
  PRIMARY KEY (item_id)
```

使うSQL（写経してから実行）

-- 1) 見る（まず確認）

```
SELECT item_id, category, item_name, unit_price FROM items ORDER BY item_id;
```

-- 2) 変更する（単価を1つだけ変える）

```
UPDATE items  
SET unit_price = 110  
WHERE item_id = 101;
```

-- 3) もう一度見る（変わった？）

```
SELECT item_id, category, item_name, unit_price FROM items ORDER BY item_id;
```

-- 4) 消す（1行だけ消す）

```
DELETE FROM items  
WHERE item_id = 202;
```

-- 5) 最後に見る（減った？）

```
SELECT item_id, category, item_name, unit_price FROM items ORDER BY item_id;
```

▼ ヒント

- 最初は `SELECT *` で「見える化」してから始める。
- `UPDATE` / `DELETE` は **WHERE (id指定)** を必ず付ける。

▼ 解答例

【解説・確認用】

正解は「変わった／消えた」を確認できることです。WHEREなしのUPDATE/DELETEは危険なので、必ずid指定を付けます。

```
SELECT item_id, category, item_name, unit_price FROM items ORDER BY item_id;  
UPDATE items SET unit_price = 110 WHERE item_id = 101;  
SELECT item_id, category, item_name, unit_price FROM items ORDER BY item_id;  
DELETE FROM items WHERE item_id = 202;  
SELECT item_id, category, item_name, unit_price FROM items ORDER BY item_id;
```

5. まとめ（写経で締める）

SELECT = 取り出して見る（結果は表示用の表）

INSERT = 1行追加（次のSELECTで増えた分が見える）

UPDATE = 1行変更（WHEREで狙って変える）

DELETE = 1行削除（WHEREで狙って消す）

DB = 最後に残る箱（消えない）